

My OpenProof project and demo guide

My explanation of the project, how I demonstrate it, and the checks I use to assess it. Release notes and recorded results: 27 September 2026.

The problem I am addressing

Imagine a Lagos vendor says, "I will dispatch your order by Friday." Reading that message tells me what was promised. Checking the dispatch record tells me what happened. I need both before I can fairly compare the promise with the result.

I use the same idea in OpenProof, with a specific focus on blockchain records. I turn "Did this project do what it said?" into a precise question: which contract, which field, what expected value, and what deadline? OpenProof does not check deliveries or bank transfers; those are familiar ways to explain why a claim and evidence are different things.

What my project checks

I offer two modes. In **Public promise**, I need an accessible source establishing the exact dated promise as well as the relevant blockchain record. In **Contract state**, I simply check a historical value without suggesting that anyone made a promise.

For a Nigerian Web3 community, I might illustrate a treasury deadline using an ajo or esusu group's agreed payout date. If the group used a supported smart contract, I could compare its `unlockTime()` value with the stated deadline. This is a hypothetical example: OpenProof does not access contribution accounts or verify that every withdrawal route is blocked.

My template	What I compare	What that comparison cannot establish
Admin and owner control	<code>owner()</code> against an expected address	Every admin role or who ultimately controls it
Treasury restrictions	<code>unlockTime()</code> against a minimum timestamp	Continuous locking or every withdrawal path
Contract changes	EIP-1967 implementation against an expected address	Changes using other proxy standards

I collect blockchain evidence from Ethereum or Base. GenLayer Bradbury, chain 4221, is where the decision is made. Submission uses test GEN for network gas; my app does not trade tokens or send a token payment to the Intelligent Contract.

My project links

- Application: <https://openproof-three.vercel.app>
- Source repository: <https://github.com/Inclover05/openproof> (private).
- Vercel project: <https://vercel.com/inclover05s-projects/openproof>
- Guide: <https://openproof-three.vercel.app/OpenProof-Project-and-Demo-Guide.pdf>

How I take a question through OpenProof

1. I define exactly what I want to check. I enter a neutral statement, select a mode and template, and specify the contract, Ethereum or Base, a past cutoff, and an expected value. I use UTC in the form. For example, 08:00 UTC is 09:00 WAT in Nigeria. The app rejects future cutoffs and unsupported chains or fields.

2. I check whether the source is readable. In Public promise mode, I can use supported direct public X posts, readable Medium articles, GitHub sources, verified explorers, and selected official documentation. "Check link access" gives me an early retrieval result. I treat that as an access check, not a verdict on the claim.

3. I review the evidence. The server locates the finalized block at the cutoff and checks that the following block falls after it. It reads the selected contract field at that historical block. I can inspect the source URL, retrieval time, access status, and relevant content hash, block, chain, and value. The preview lasts 15 minutes.

4. I save the case. I can collect and save without connecting a wallet. The saved inputs must match the evidence snapshot. Saving the same snapshot twice returns the same case. At this point, I have a draft with no verdict. "Create revised case" lets me start a new case from the old inputs while retaining the original record.

5. I request a network decision. I connect a browser wallet to Bradbury and review a fresh fee estimate. The app checks the account and network before asking me to sign the exact payload. It retains the transaction ID locally and on the server so I can recover tracking after a timeout.

6. I inspect what the network established. GenLayer validators independently retrieve the reviewed sources, compare their hashes, check historical contract state, and apply the contract's rules. The contract requires strict agreement on the decision fields. My website displays the result; it cannot choose the final outcome itself.

Result I see	How I explain it
Supported	The eligible evidence supports this exact comparison
Contradicted	The eligible evidence conflicts with this exact comparison
Insufficient evidence	A fact needed to decide the question could not be established

I also check execution and finality separately. An accepted transaction can fail during execution. A successful decision can still be in its appeal window. My interface displays appeal availability when the network provides it, but this release has no public appeal-signing button.

How I present my five minute demo

My preparation

I keep the homepage and the three saved cases below open before I begin. These give me real recorded outcomes even if the testnet is slow. For a new submission, I need a browser wallet with test GEN on Bradbury. OpenProof never needs my seed phrase or private key. I stop and investigate if my wallet shows a security warning.

Minute one: I explain the idea

"Imagine an ajo group agrees that its funds should remain locked until a particular date. If that agreement uses a supported smart contract, I want to compare the exact promise with the relevant blockchain record. That is the kind of narrow question I built OpenProof around. I can review the evidence before connecting a wallet, then ask GenLayer to evaluate it independently."

I show "What can you actually prove?" on the homepage and switch between the three outcomes. I explain that this interactive section is a labelled illustration. A missing fact means there is not enough evidence to decide; it does not automatically mean somebody broke a promise.

Minute two: I show a real matching result

I open "Matching owner." This is a historical Base USDC contract-state test. I am not claiming that Circle made a promise. I show the contract, cutoff, block 51,853,326, observed owner, transaction, successful execution, and lifecycle. The recorded outcome is **Supported**.

<https://openproof-three.vercel.app/case/2735f336-d5cf-417c-a267-8f0918455293>

Minute three: I change what is being compared

I open "Different owner." The same historical owner was compared with the zero address, so this result is **Contradicted**. I explain that a failed comparison does not establish dishonest intent or tell me whether a token is safe.

<https://openproof-three.vercel.app/case/d7ca8a0d-92cd-4d16-b68d-9ba999205c4d>

Minute four: I show why a readable link is not enough

I open "Unrelated X source." The post could be read, but it did not establish the submitted contract promise. The result is **Insufficient evidence**. It is like showing a genuine dispatch receipt for a different Lagos order: the document may be real and still answer the wrong question.

<https://openproof-three.vercel.app/case/72f7f19d-e0b1-488f-9ace-a06567b0c37b>

Minute five: I create a draft

I select "Create revised case" from the matching-owner record, review the inputs, collect evidence, and save. I point out that a draft has no verdict. If time permits, I review the testnet fee and submit. Otherwise, I finish with the completed cases rather than depend on an instant response.

My closing explanation: "OpenProof gives me a record of the question, the sources, the historical observation, and what the network could establish. I want someone reviewing my result to be able to inspect that evidence and understand its limits."

My reproducible historical state tests

For these tests, I select **Contract state**, **Base**, and **2026-09-27 08:00 UTC**. That is **09:00 WAT on 27 September 2026** in Nigeria; I still enter 08:00 in the UTC field. I leave the source blank and use the statement provided below.

I collect evidence, inspect the value, and save. Saving alone should show **Draft**, with no outcome. The expected decisions below apply after successful network evaluation with the required evidence available.

Test one: My expected owner matches

- Template: Admin and owner control.
- Contract: 0x833589fCD6eDb6E08f4c7C32D4f71b54bdA02913
- Field: owner()
- Expected address: 0x3abd6f64a422225e61e435bae41db12096106df7
- My statement: Historical comparison test Base USDC owner equals the expected address at the cutoff.
- Expected decision: **Supported**.
- Recorded result: Supported, successful execution, five AGREE votes.
- Saved case: <https://openproof-three.vercel.app/case/2735f336-d5cf-417c-a267-8f0918455293>

Test two: My expected owner differs

I repeat test one and change only the expected address to **0x00000000000000000000000000000000**. I expect **Contradicted** because the recorded historical owner is the nonzero address above. The release test returned Contradicted, with five AGREE votes. I use this as a comparison test, not an allegation about the project.

Saved case: <https://openproof-three.vercel.app/case/d7ca8a0d-92cd-4d16-b68d-9ba999205c4d>

Test three: My expected implementation matches

- Template: Contract changes.
- Contract: 0x42000
- Field: EIP-1967.
- Expected address: 0xc0d3c0d3c0d3c0d3c0d3c0d3c0d3c0d3c0d3c0d3c0d30010
- My statement: Historical comparison test Base bridge implementation equals the expected address at the cutoff.
- Expected and recorded decision: **Supported**, with five AGREE votes.
- Saved case: <https://openproof-three.vercel.app/case/62321760-00ca-4f50-bf31-eaf3dba2594e>

All three recorded observations used Base block 51,853,326. Public blockchain data services can become unavailable. If a later run cannot independently obtain the historical value, I expect an insufficient-evidence or failure state. I do not replace the historical value with today's value just to produce the expected answer.

My functional test checklist

What I try	What I expect to see
I enter a future cutoff	Validation rejects it before collection
I enter an incomplete contract address	A useful error, with no submission
I omit the Public promise source	A source-required message
I omit the Contract state source	Allowed; historical chain evidence is still required
I use https://127.0.0.1/admin as a source	Blocked without fetching the private address
I use a redirecting source	Blocked; I need the final supported URL
I try an unsupported getter such as <code>admin()</code>	Rejected by the supported input rules
I change inputs after collection	I must collect a fresh snapshot before saving
I save the same snapshot twice	The same case ID, without a duplicate record
I reload my saved case	My original inputs and evidence persist
I create a revised case	A prefilled builder; my original record remains intact
I attach an unrelated or invalid transaction	Rejected; the contract and payload must match
I open the app without a browser wallet	Reading works; signing reports no wallet
I change account or chain before signing	I need to reconnect and obtain a fresh quote
I lose tracking after submission	I can resume with the retained transaction ID
I open a case at a 390 pixel phone width	Readable controls and no horizontal overflow

My release's recorded coverage

The release checks recorded **30 passing direct contract tests**, **11 passing JavaScript regression tests**, and **12 passing API acceptance checks** through the running app and Neon database. Those API checks covered persistence, tamper rejection, request-origin rejection, source access, and live fee quoting. Production build, lint, and type checks passed.

Browser verification covered desktop rendering, the complete collect-review-save journey, mobile case display and reload, the interactive explanation, and prefilled revision. No browser errors were reported in those checked journeys. Production check records are in `reports/release.json` and `reports/api-verification.json`.

I distinguish automated fixtures from live network tests. Wallet-extension signing on the final public domain, a public appeal submission, sustained load, and a positive live treasury promise were not fully exercised. The recorded live transactions used a funded local testnet signer. These results describe completed release checks, not every possible scenario.

What runs behind my website

My architecture in plain language

My Next.js website is the place to enter a question, review evidence, save a case, and interact with a wallet. Its server checks inputs and gathers the preview. Neon Postgres is the database that retains case inputs, snapshots, and request-limit counters. GenLayerJS connects the app to fee estimates, submissions, and transaction status.

The Python Intelligent Contract performs the network evaluation. It retrieves the reviewed evidence and historical blockchain observations independently, then records the outcome. A content hash acts like a fingerprint: it helps detect whether reviewed source text changed between collection and evaluation.

For X and Medium, version 2 normalizes the relevant text before hashing so irrelevant page wrappers do not change that fingerprint. Other raw sources retain raw-content hashes. Changed reviewed content leads to insufficient evidence. My server uses a fixed supported-host policy, does not silently follow redirects, and caps source bodies at 500 KB and JSON requests at 10 KB.

My main contract address

My current main OpenProof contract is version 2 on GenLayer Bradbury:

0xB74B3339695C50C6d16168708e2B22A7D6D72EAB

I can read it here:

<https://explorer-bradbury.genlayer.com/address/0xB74B3339695C50C6d16168708e2B22A7D6D72EAB>

- Bradbury chain ID: 4221; wallet chain hex: 0x107d.
- Legacy OpenProof contract: 0x0546Ba4582b7733DB52E3309692BCF7b5B1CAcCe.
- SDK: genlayer-js 1.1.8.
- V2 deployment transaction:
0x5ddcfa3a55765f0afe6053e6fae5fbb67294e3db38acac74bc0d62a5374d23b

I retain the legacy address for older records and their original payload format. Version 2 records identify their protocol version explicitly. The Base USDC and bridge addresses in my tests are contracts I inspect; they are not additional OpenProof decision contracts.

My fees and finality explanation

The recorded v2 deployment balance difference was about 0.001254839 test GEN. An earlier isolated v1 judgment cost about 0.000163473 test GEN. I use these as historical examples, not fixed prices. The app obtains a fresh quote before confirmation, valid for two minutes. The test signer used no real-money funds.

All four new judgments reached successful finalization in the recorded production checks. One unrelated-source run included a validator timeout. An earlier v1 negative case is also finalized. I check the lifecycle on the live record rather than assume that seeing an outcome means the appeal window has ended.

How I maintain the project and explain its limits

My deployment routine

For a code release, I install dependencies with `npm ci`, link the existing Vercel project, and pull environment variables into ignored `.env.local`. If a database migration is required, I run `npm run db:migrate` against the intended database. I run `npm test`, `npm run lint`, and `npm run build`. My GitHub main branch deploys to Vercel; I then inspect the deployment and check the production API and browser.

I keep `DATABASE_URL` in server environment configuration. Vercel does not need the local test signer's private key. That signer stays in ignored `.secrets`, outside the website runtime, frontend code, repository uploads, and this guide.

What I do if a demo step fails

- **Blocked source:** I show the access result and, where available, add a relevant corroborating source. If I only want to check a historical value, I create a Contract state case with that different scope.
- **Historical data unavailable:** I retain the cutoff, acknowledge the missing observation, and use a saved assessed case or retry later.
- **Pending transaction:** I refresh tracking or attach the retained transaction ID. I check wallet history before considering another submission.
- **Storage failure:** I retain my inputs and check Vercel configuration and Neon availability. If the snapshot expired, I collect fresh evidence.
- **Rate limit:** I wait five minutes. The app allows ten requests per action per IP window, so people sharing a connection can share that allowance.

The boundaries I make clear

I present OpenProof as a beta for specific evidence questions. It does not determine motives, identity, beneficial ownership, every admin role, continuous treasury restrictions, or all upgrade mechanisms. It is not a general fact checker, legal determination, or security audit. A source visible today does not independently prove the same text was published before a historical cutoff.

My Nigerian examples explain the idea; they do not mean the app connects to banks, ajo accounts, delivery companies, or government records. The live tests use the specified blockchain contracts and public links. Public data providers and testnet services can fail or limit requests.

Cases on my public deployment can be read by anyone with their URL, so I keep private information out of them. My source repository is private; reviewers need access. My original Sites deployment remains at <https://openproof-evidence.inclover05.chatgpt.site>. Only labelled public test cases were copied into Vercel, and old Sites case URLs do not automatically become Vercel case URLs.

My technical references

- GenLayer web access: <https://docs.genlayer.com/developers/intelligent-contracts/features/web-access>
- GenLayer transactions: <https://docs.genlayer.com/understand-genlayer-protocol/core-concepts/transactions>
- Circle contract addresses: <https://developers.circle.com/stablecoins/usdc-contract-addresses>
- Base contract reference: <https://docs.base.org/specifications/reference/base-contracts>

I keep transaction IDs, evidence values, provider observations, and verification scripts in the repository so my technical claims can be traced to records. A Builder portal submission is a separate publishing step.